IMAGE PROCESSING WITH PYTHON

COURSE: ITT440
CLASS: M3CS2554A
BY: MUHAMMAD ABDULLAH BIN JOHARI
STUDENT ID: 2025231878

1.0 Introduction

Image processing is commonly used in many applications, such as photo editing, social media, websites, and digital photography. It lets the users to modify or enhance the pictures using various methods. Examples include blurring, converting images to grayscale, and resizing images. For this reason, Python is one of the programming languages that can be used to do these things since there are libraries in it that are useful to handle and process images.

This project was to create a web app that does simple image processing in python. There are three types of operations included in the application which are Gaussian Blur, Grayscale, and Thumbnail Resize. Users can upload an image and select one of the three operations. Flask is used to process user requests and Pillow is used to process images. The user uploads an image on this web page, chooses an effect and can see the result.

An application that does work correctly doesn't necessarily mean that it can support many users at a time. The higher the number of users visiting a particular website, the more requests the server must handle. This can result in slower response times, increased CPU and memory consumption, or even requests being denied. As a result, performance testing is essential to determine the behavior of an application in various situations.

For this reason, I used Locust as the performance-testing tool for this project. Using Locust, I can simulate multiple users accessing my application and gather some key performance metrics like response time, throughput, and failure rate. The tests were conducted in three categories: load testing, stress testing and soak testing.

The aim of this project is to investigate the performance of an image processing application called Python with varying numbers of users accessing it. The test results can also be used to determine which image processing operation is the slowest and what improvements can be made to the application.

2.0 Tools Selection Justification

For this project, Apache Locust was chosen as the performance testing tool. Locust is an open-source performance-testing tool that can create more than one user to access the web application. It was selected because it uses Python to create test scenarios, which makes it suitable for a project that is already developed using Python.

The simplicity of testing setup is one of the major factors behind choosing Locust. Users can develop Python scripts to manage the simulated users' interactions with the application instead of using a complicated interface to create all the tests scenarios. This will make it easier to experiment with different image processing functions, like blurring, converting to grayscale and resizing to thumbnails.

One more advantage of selecting Locust is that it offers beneficial performance numbers. It shows the number of requests, number of failed requests, response times and requests per second. These measurements give me an idea of whether or not the application can successfully serve requests as the number of users grows.

Locust also provides you with the option to test in various scenarios. It was used in this project for load testing, stress testing and soak testing. There are different purposes in each test. Load testing is a test of the application under an expected workload, stress testing is a test that pushes the application to a higher workload to see how it performs and soak testing is a test of the application's performance over an extended amount of time.

In summary, Locust is a good option for testing since it can run Python scripts, offers valuable performance reporting and enables various performance tests to be carried out with the same tool.

3.0 Test Environment Setup

It was developed with Python, Flask and Pillow. The communication between the website and the Python backend is handled by Flask, and the image-processing operations are carried out by Pillow. The application includes three major image-processing capabilities:

Gaussian Blur: This function will blur the uploaded image. It employs Pillow's Image Filter.Use GaussianBlur() function with a radius value of 2. The effect blurs the image and creates softer details.

Grayscale Conversion: This function will convert an image from the color space to gray scale. It uses Pillow's ImageOps.grayscale() function. This will give you an image in black and white, with the colour information lost.

Thumbnail Resizing: This process cuts the size of the image that has been uploaded. It utilizes Pillow's thumbnail() function, with a maximum size of 800x800. It also preserves the proportions of the original image so that it does not get stretched.

The test environment was a Windows computer running Python 3.15.2. The necessary python libraries were installed from requirements.txt. Additionally, a virtual environment

(called .venv) was set up to isolate the project's Python packages from other packages installed in the system.

The application was started using the command python app.py. It was used as the local address http://127.0.0.1:5000. The same Python environment was used to install Locust and then use it to send requests to the Flask application.

The test environment included the following software:

COMPONENT	DESCRIPTION
Programming language	Python 3.15.2
Web framework	Flask
Image-processing library	Pillow
Performance-testing tool	Locust
Operating sytem	Windows
Test Target	Flask image-processing application
Target address	127.0.0.1:5000

A full description of the test environment should include the hardware model, processor specifications, and RAM capacity installed and will be included in the final version of this article. A major drawback to this was that the application and Locust were both installed on one machine. Thus, the results of this test are not a production server but are merely the results of the local test environment.

4.0 Methodology

There are three main performance tests that were conducted using Locust. Before conducting these tests, a short smoke test was performed to check whether the application could receive and process requests correctly.

4.1 Smoke Test

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /health	5	0	5.14	5	6	16	0.17	0
POST	POST /api/process [blur]	12	0	189.87	181	213	70950	0.42	0
POST	POST /api/process [grayscale]	5	0	54.18	48	61	23580	0.17	0
POST	POST /api/process [thumbnail]	7	0	106.07	101	127	60739	0.24	0
	Aggregated	29	0	114.4	5	213	48088.03	1	0

Smoke test locust test report

The smoke test was the first test conducted. It was set up with two simulated users and tested for 30 seconds. The aim was to verify the responsiveness of the application and its functions related to image processing.

A total of 29 requests were generated by the test and no failures were reported. The average response time was 114.4 milliseconds. The outcome indicated that the application was performing properly with a light load. This test was useful in verifying the application before subjecting it to the more challenging tests for performance.

4.2 Load Test

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /health	229	0	6.13	4	20	16	0.77	0
POST	POST /api/process [blur]	397	0	220.63	168	302	70950	1.33	0
POST	POST /api/process [grayscale]	379	0	60.23	52	198	23580	1.27	0
POST	POST /api/process [thumbnail]	398	0	123.1	90	232	60739	1.33	0
	Aggregated	1403	0	114.62	4	302	43679.08	4.7	0

Load test locust test report

The load test was performed to assess the performance of the application when increased number of users use the app simultaneously. It was pretty simple, with 10 simulated users and a five minute runtime. Gradual increase of users by 2 per second, until they reach the maximum number in the "maxUsers" setting.One by one, gradually add users at a rate of two per second till the maximum in the "maxUsers" setting is reached.

During the test, Locust made calls to the health-check and image-processing points of the application. The results were presented for Response Times, Total Requests, Failed Requests and Throughput.

There were 1,403 requests recorded where there were no failures. The overall average response time was 114.62 milliseconds. The mean throughput was 4.70 requests/second.

4.3 Stress Test

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /health	265	0	6.36	5	19	16	1.1	0
POST	POST /api/process [blur]	387	0	220.29	197	279	70950	1.61	0
POST	POST /api/process [grayscale]	367	0	59.34	51	76	23580	1.53	0
POST	POST /api/process [thumbnail]	380	0	123.48	108	171	60739	1.58	0
	Aggregated	1399	0	111.25	5	279	42313.49	5.82	0

Stress test locust test report

The stress test was conducted to observe the application's behaviour when the number of users increased. The intent of this test was to explore the application's behavior under increased traffic, which was not the goal of load testing.

The test was conducted with the number of simulated users gradually increased up to 20 users. The workload increased at various stages, culminating in the highest.

During the test, Locust made 1,399 requests without any reported failures. The average value of the response time was 111.25 msec. The average throughput is 5.82 requests per second.

The results indicated that the application was able to keep responding under the test stress. But because there were no recorded request failures, the test didn't determine exactly when the application would start to fail.

4.4 Soak Test

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /health	720	0	6.2	5	106	16	0.4	0
POST	POST /api/process [blur]	1213	0	211.45	151	400	70950	0.67	0
POST	POST /api/process [grayscale]	1191	0	56.75	43	77	23580	0.66	0
POST	POST /api/process [thumbnail]	1127	0	118.41	82	165	60739	0.63	0
	Aggregated	4251	0	108.68	5	400	42957.07	2.36	0

Soak test locust test report

The soak test was done to check if it can perform consistently over longer periods of time and with 4.4. It was set up to simulate 5 users for 30 minutes.

In this timeframe, Locust was able to process 4251 requests and reported no failures. The average response time was 108.68 milliseconds. The mean throughput was 2.36 requests per second.

5.0 Raw Data Presentation and Result

Operation	Total Request	Failed Request	Average Response Time
Gaussian blur	1,213	211.45ms	240ms
Grayscale	1,191	56.75ms	66ms
Thumbnail	1,127	118.41ms	130ms
Overall	4,251	108.68ms	220ms

According to the results of soak test, all three image-processing operations were able to process their requests without any failures reported. This means that the app was able to process images throughout the 30-minute test.

The quickest average response time was in the grayscale conversion with 56.75 milliseconds. The average response times for the thumbnail resizing and Gaussian blur were 118.41 milliseconds and 211.45 milliseconds respectively.That would imply Gaussian blur would take about 3.7 times more time to process than converting to grayscale. Hence, Gaussian blur is the most critical for future performance improvement.

Overall, the results showed that the application has a good response time for 30 minutes during the testing.

6.0 Recommendations and Conclusion

Finally, based on the performance testing results, some enhancements can be suggested for the image-processing application. The first thing is to pay more attention to the Gaussian blur function as it had the highest average response time of 211.45 milliseconds. The processing method and blur settings can be checked to determine if it is possible to make this operation faster.

Secondly, the application should keep on validating the sizes of the images uploaded by the users, so that they won't upload a large sized one. This can help to limit the amount of necessary processing time and memory. Third, when testing the application's performance in the future, the use of the CPU and memory should be monitored to gain a deeper understanding of the application's computer resource consumption. The production-ready web server should also be taken into account before making the application public.

To sum up, this project proved to be a successful implementation of Python, Flask, Pillow, and Locust for creating and stress-testing an image-processing web application. The application has 3 primary functions: Gaussian blur, grayscale, and thumbnail resize. There was no reported failure during the performance tests for the application to process the test workloads. There were no failures on the 30-minute soak test, which yielded 4,251 requests.

Among the three image-processing functions, grayscale conversion was the quickest, while Gaussian blur was the slowest. Overall, this project was a great way for me to learn about performance testing and how it can be used as a method of testing a web application to see if there are any areas where it can be improved. The results were promising, but needed further testing using a public deployment and appropriate CPU and memory monitoring to gain insight into the performance of the application.

7.0 References and Video Demonstration

Tools and documentation:

1. Locust Documentation: https://docs.locust.io/
2. Flask Documentation: https://flask.palletsprojects.com/
3. Pillow Documentation: https://pillow.readthedocs.io/

YouTube Demonstration: Image Processing With Python